

AlgoAgentX

Algorithmic Trading SaaS Platform

Backtesting

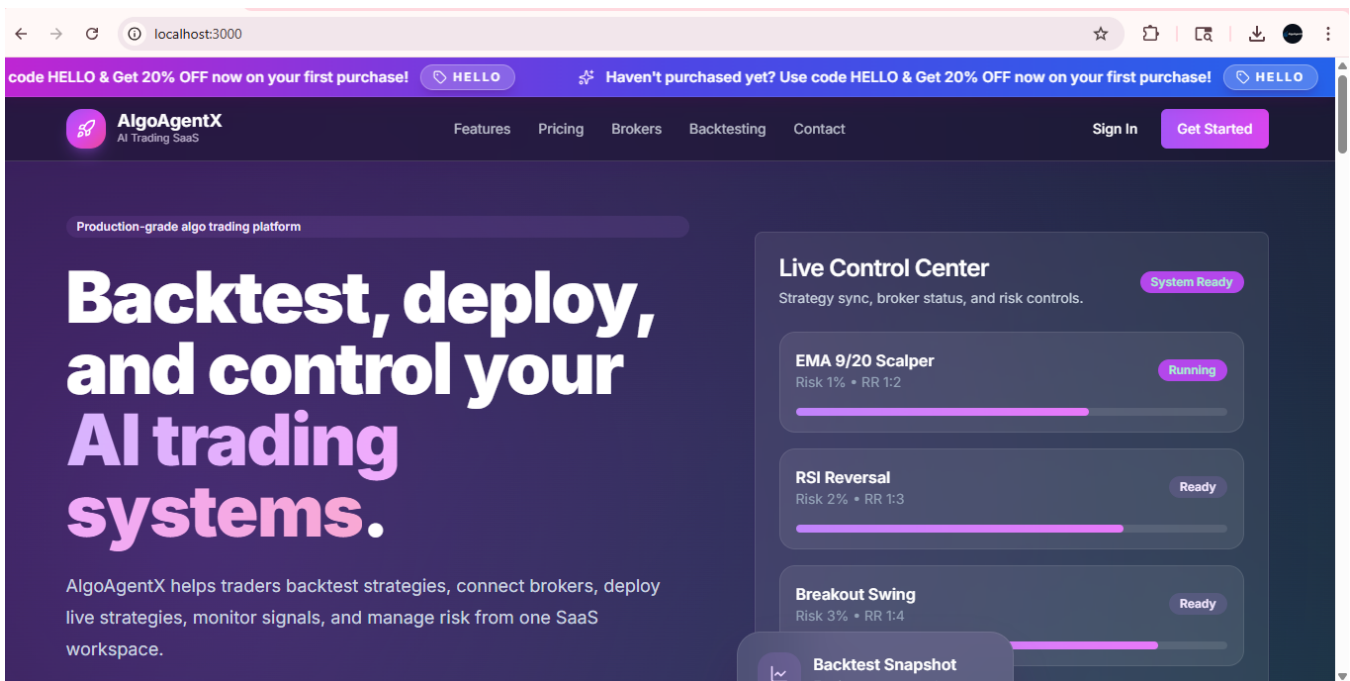
Live Trading

Broker APIs

Risk Engine

Admin SaaS

A product case study for an end-to-end trading platform covering strategy testing, runtime trade management, broker connectivity, live execution readiness, reporting, billing, and platform administration.

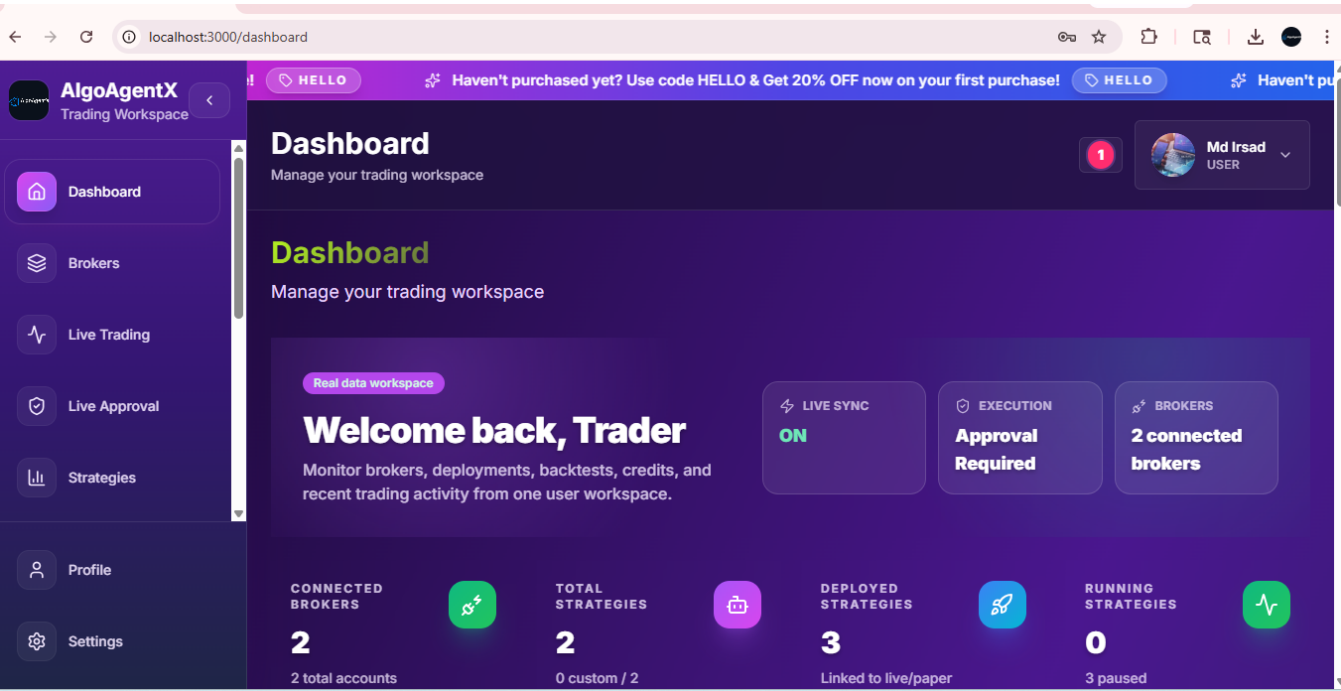


Home page / product landing view

Prepared by Md Irsad

1. Product Overview

AlgoAgentX is an algorithmic trading SaaS platform designed to help users configure strategies, run structured backtests, compare trade-management approaches, validate data coverage, connect brokers, and prepare live deployments with risk controls.



User workspace with platform navigation and module access

Core platform modules

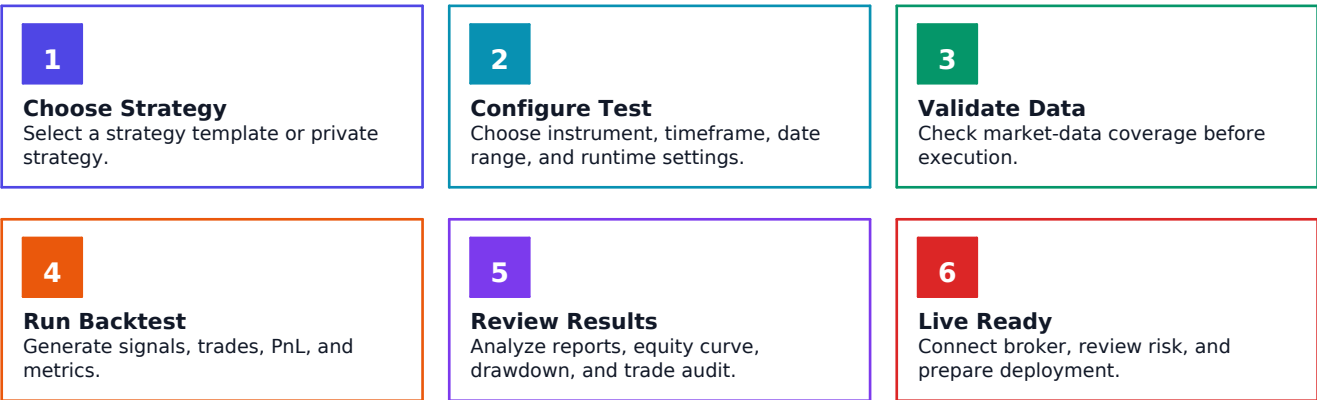
Strategy Workspace Public and private strategy templates with user-facing configuration.	Backtesting Instrument, timeframe, date range, runtime settings, coverage checks, metrics, reports.	Live Trading Broker connection, deployment settings, runner status, candle-close execution flow.
Broker Layer Upstox-style cloud broker connectivity and MT5 Agent architecture.	Admin Console Market data, instruments, strategies, users, approvals, billing, support, settings.	SaaS Operations Plans, credits, coupons, orders, payments, subscriptions, and usage controls.

2. Problem and Solution

Trading strategy testing often becomes fragmented across spreadsheets, scripts, broker terminals, and manual notes. AlgoAgentX brings the main workflow into one controlled platform where strategy configuration, historical validation, live execution readiness, and SaaS operations are handled in a single product structure.

Problem	Solution
Strategies are difficult to evaluate consistently when data quality, runtime settings, risk rules, and reporting are separated.	A unified workflow that connects strategy templates, validated market data, configurable risk/trade management, analytics, reports, broker connectivity, and admin operations.
Platform Goal	Engineering Goal
Allow traders and operators to test multiple configurations before live use while maintaining auditability and safety controls.	Separate frontend, backend, data, trading engine, broker routing, billing, and admin responsibilities so the product can scale cleanly.

Primary user flows

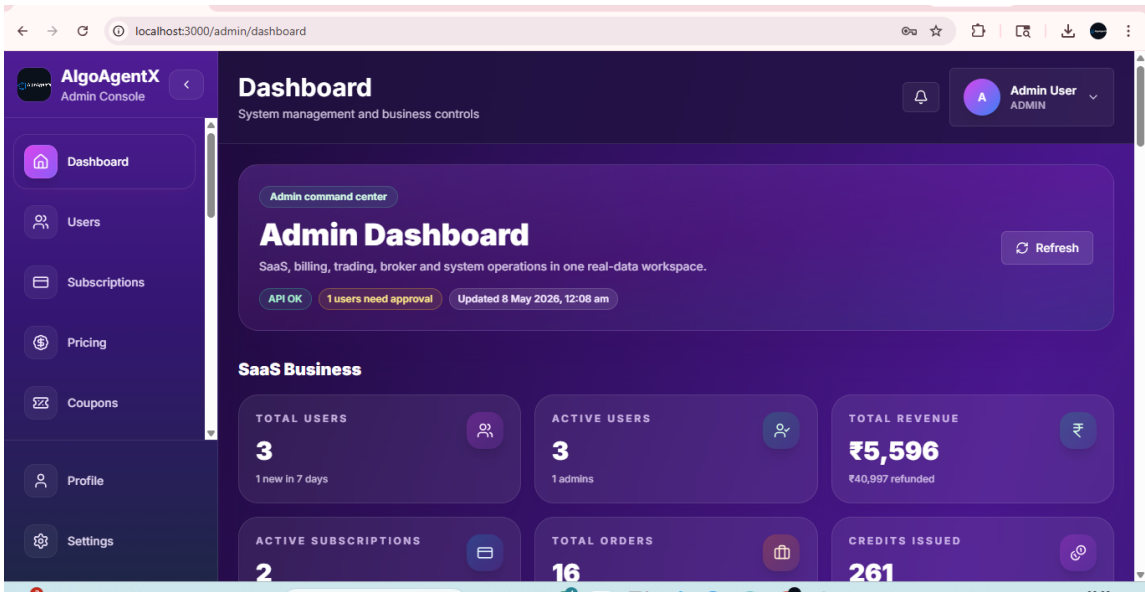
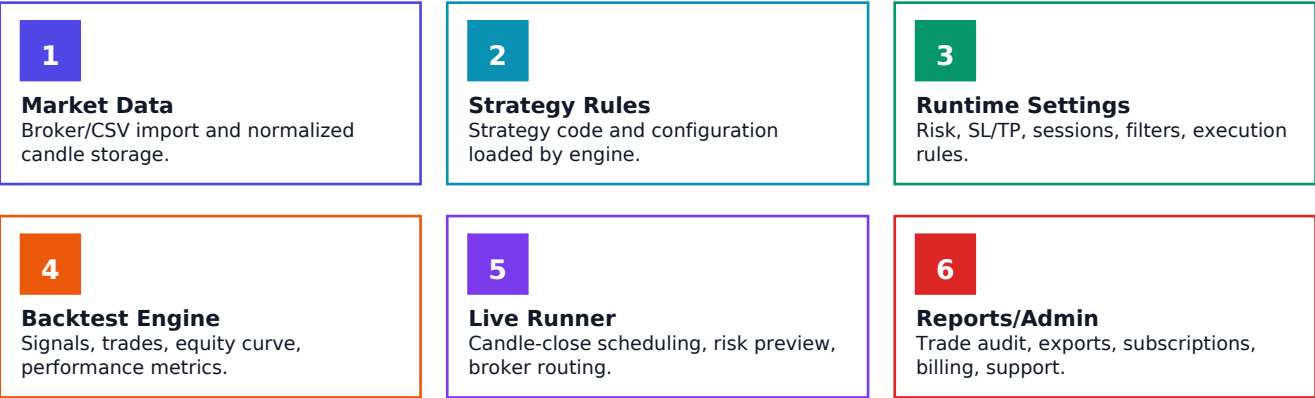


3. System Architecture

The platform is organized around a frontend workspace, FastAPI backend, trading services, PostgreSQL data layer, external integrations, and operational controls. The same runtime settings concepts are shared across backtesting and live deployment workflows.

Frontend	API Layer	Core Trading Services
Next.js, React, Tailwind CSS, user workspace, admin console, dashboards, settings, reporting pages.	FastAPI, auth, user/admin roles, validations, REST endpoints, broker routing, service orchestration.	Backtest engine, runtime settings, risk engine, signal generation, live runner, candle-close scheduler.
Data Layer	Integrations	Operations
PostgreSQL, async SQLAlchemy, market data, trades, metrics, users, subscriptions, billing, audit records.	Upstox-style broker APIs, MT5 Windows Agent workflow, market data import, payment gateway integration.	Docker deployment, logging, monitoring, migrations, timeout hardening, coverage checks, execution diagnostics.

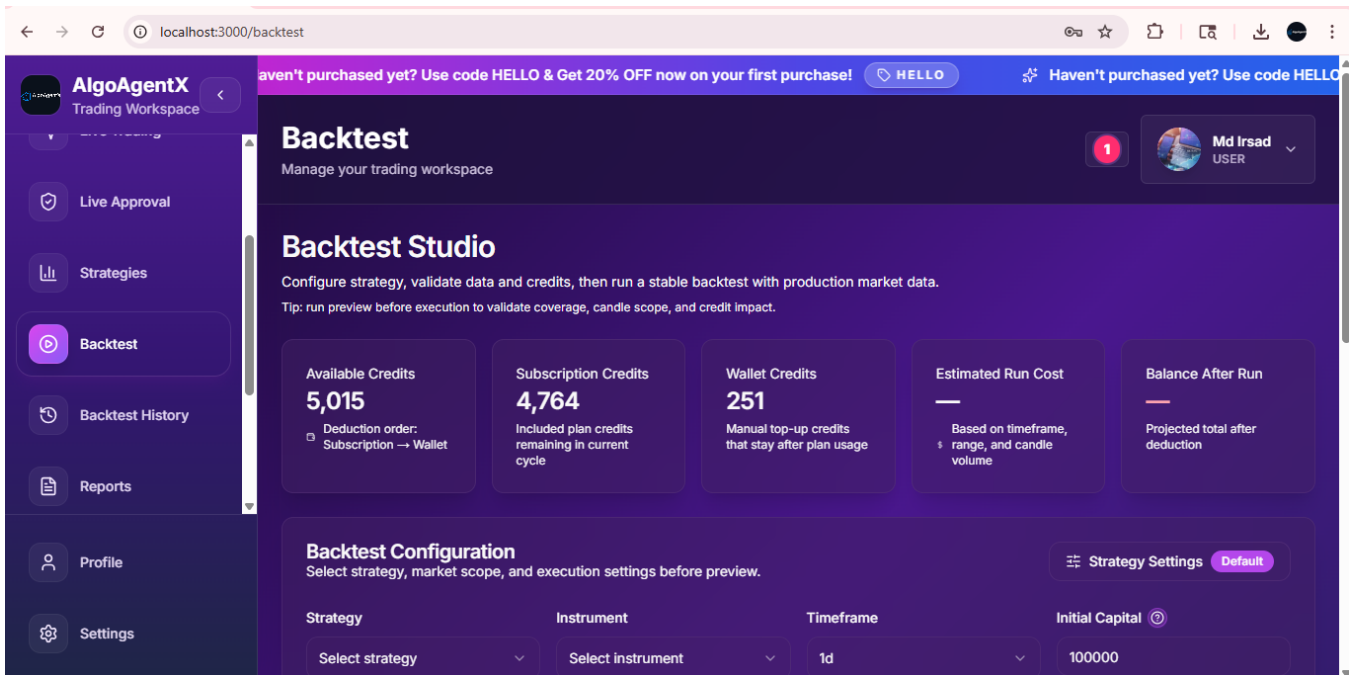
High-level data flow



Admin dashboard for operational overview

4. Backtesting Workspace

The backtesting module is designed as a controlled strategy evaluation workspace. Users can select a strategy, market, instrument, timeframe, and testing range, then apply runtime controls before running the engine.



Backtest setup page with strategy and test configuration

Backtest setup capabilities

Strategy Selection Public templates and private strategy access.	Instrument Setup Market, broker symbol mapping, timeframe, and date range.	Coverage Check Blocks execution when historical candles are incomplete.
Runtime Drawer Risk, SL/TP, sessions, day filters, and execution rules.	Cost Control Credit-based usage model for backtest execution.	Result Storage Trades, metrics, equity, drawdown, and reports saved for review.

5. Runtime Settings and Trade Management

The runtime settings layer is one of the most important parts of the platform. It allows the same strategy to be tested under different risk, SL/TP, risk-reward, session, and execution assumptions without rewriting strategy code.

Trade management controls

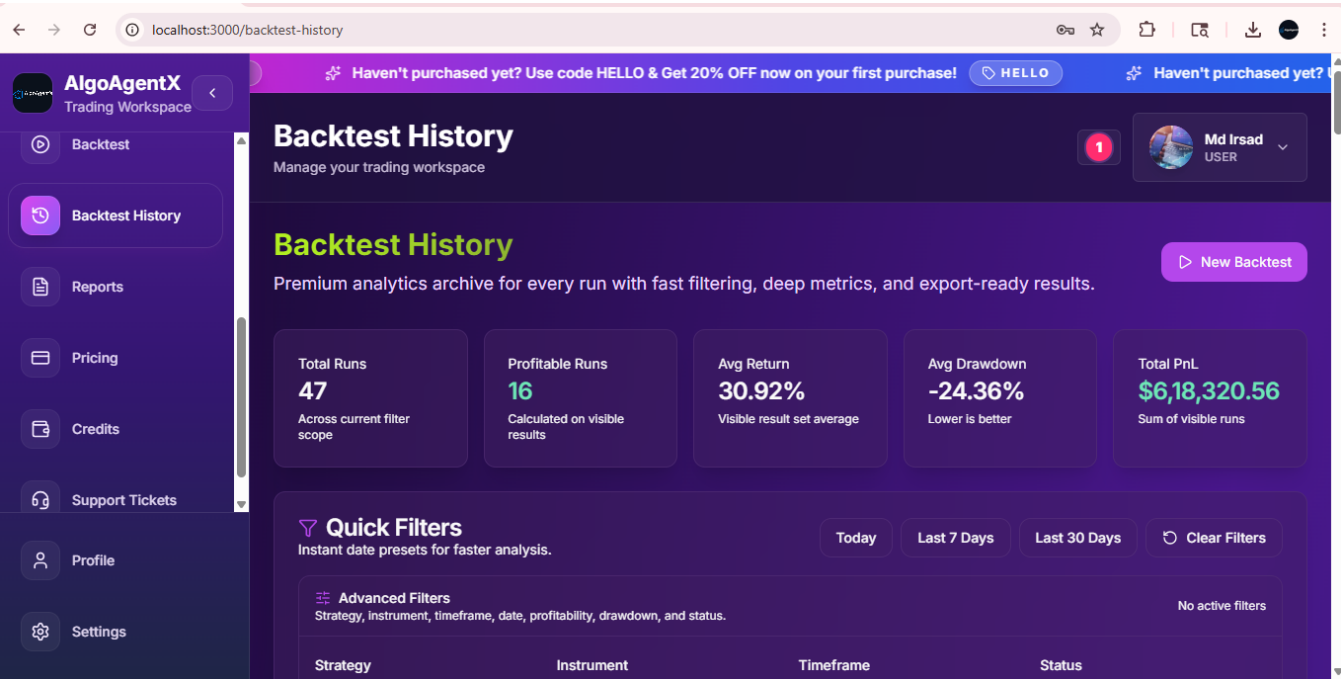
Risk Model Initial capital, risk percentage, fixed lot/quantity, account currency, and sizing rules.	SL/TP Model Swing SL, fixed percent SL, strategy-suggested SL, RR ratio, ATR and swing lookback settings.	Execution Rules Entry mode, opposite-signal exits, max trades per day, max open positions, square-off time.
Session Filters Asian, London, New York, or instrument-specific session controls.	Day Filters Selectable weekday filters to test performance by trading day.	Validation Front-end rules and backend checks prevent invalid or unsafe configuration.

Configuration test cycle

1 Base Strategy Keep signal logic constant.	2 Risk Profile Change sizing and capital assumptions.	3 SL/TP Rules Compare fixed, swing, ATR, and RR settings.
4 Filters Apply session and weekday selection.	5 Backtest Run controlled performance comparison.	6 Compare Review metrics and trade-level behavior.

6. Backtest Outputs, Analytics, and Reports

After a backtest, the platform presents performance metrics, trade history, equity progression, drawdown, and detailed trade-level information. This makes strategy review structured and repeatable.



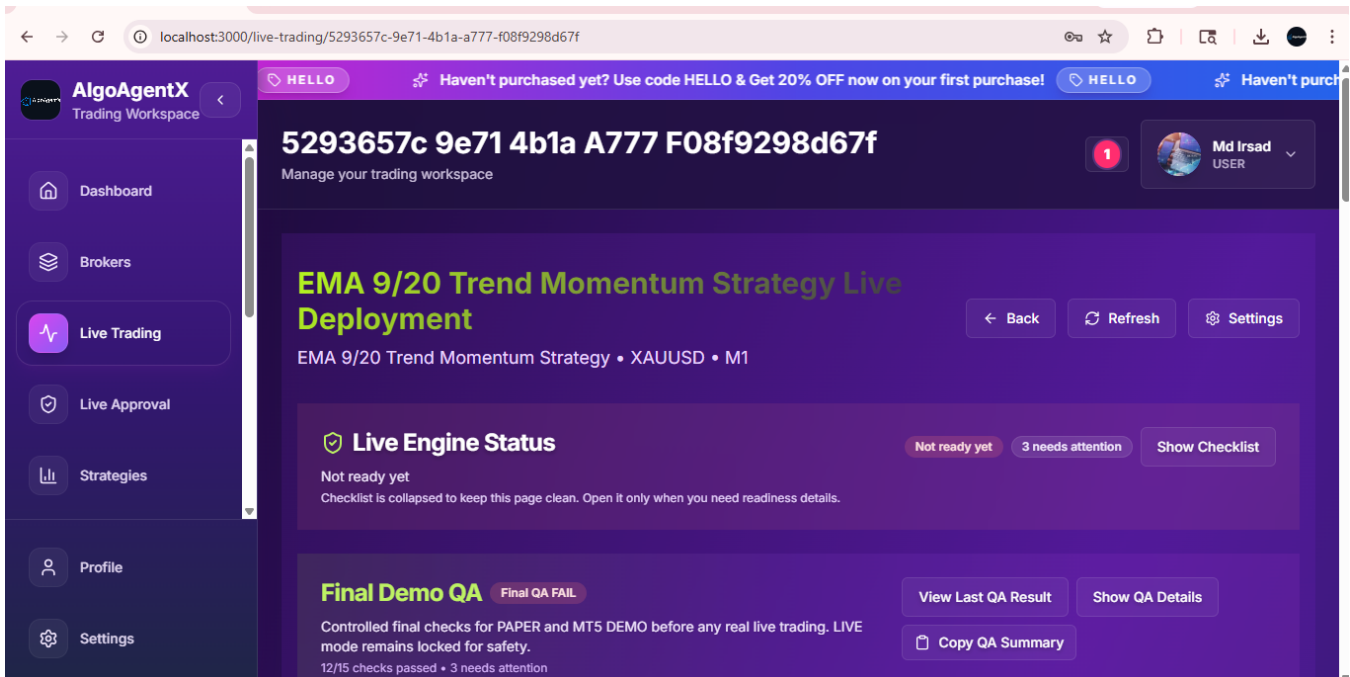
Backtest history and result review

Output and reporting features

Performance Metrics Net PnL, ROI, drawdown, win rate, profit factor, total trades, average trade.	Trade Audit Entry time, exit time, side, quantity, SL, TP, exit reason, realized PnL.	Equity Curve Visual account progression across executed trades.
Drawdown View Risk view for loss streaks and capital protection.	Exports CSV, Excel, and PDF-style reporting workflows for review.	Screenshots Product views can be used for demos, notes, and platform documentation.

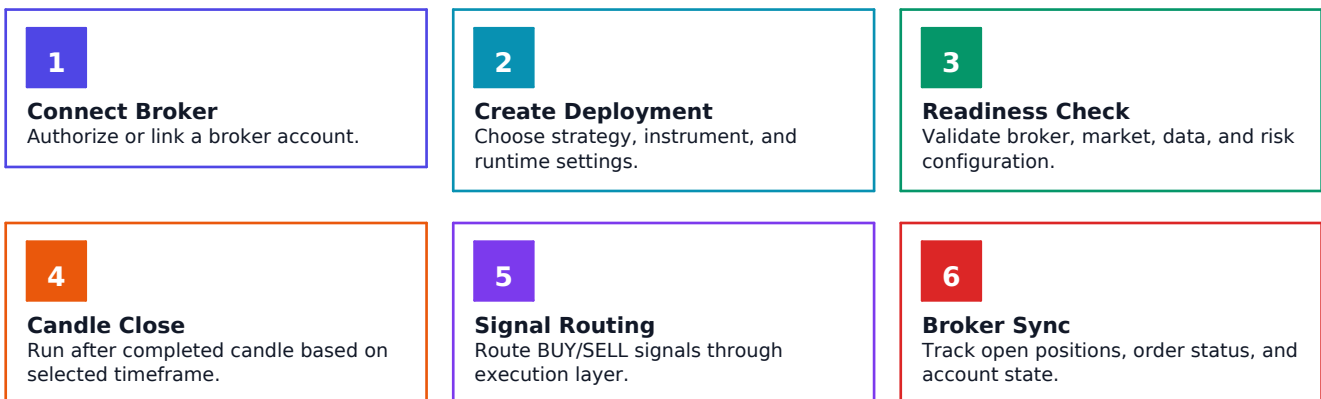
7. Live Trading System

The live trading module extends the backtesting workflow into broker-connected execution readiness. It includes broker connection, deployment settings, runner status, candle-close scheduling, risk preview, broker sync, open positions, order history, and safety checks.



Live trading workspace with deployments and execution status

Live trading workflow

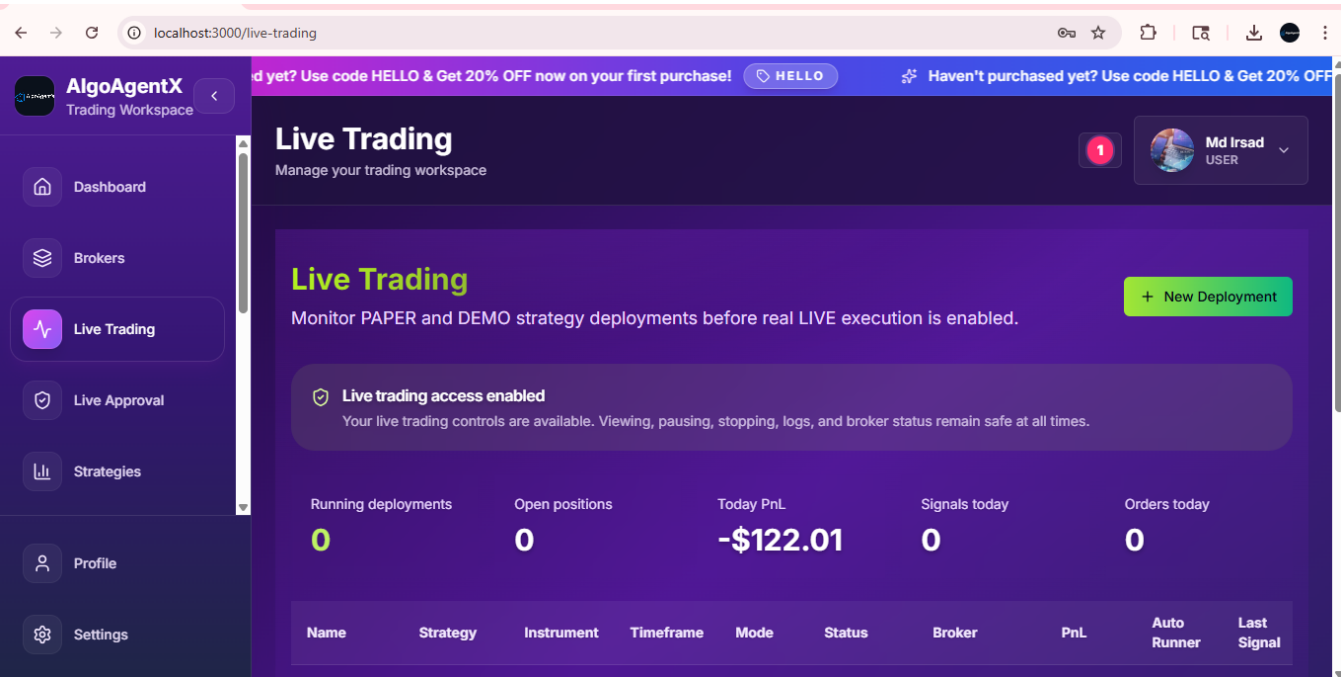


Candle-close scheduling

For a 5-minute strategy, the runner waits for the completed 5-minute candle before evaluating signals. This avoids repeated processing on incomplete candles and keeps live behavior aligned with backtest assumptions.

8. Execution Safety and Live Diagnostics

Live trading requires additional safety beyond backtesting. AlgoAgentX includes checks for readiness, risk preview, broker sync, duplicate-order prevention, and execution diagnostics before and during live operation.



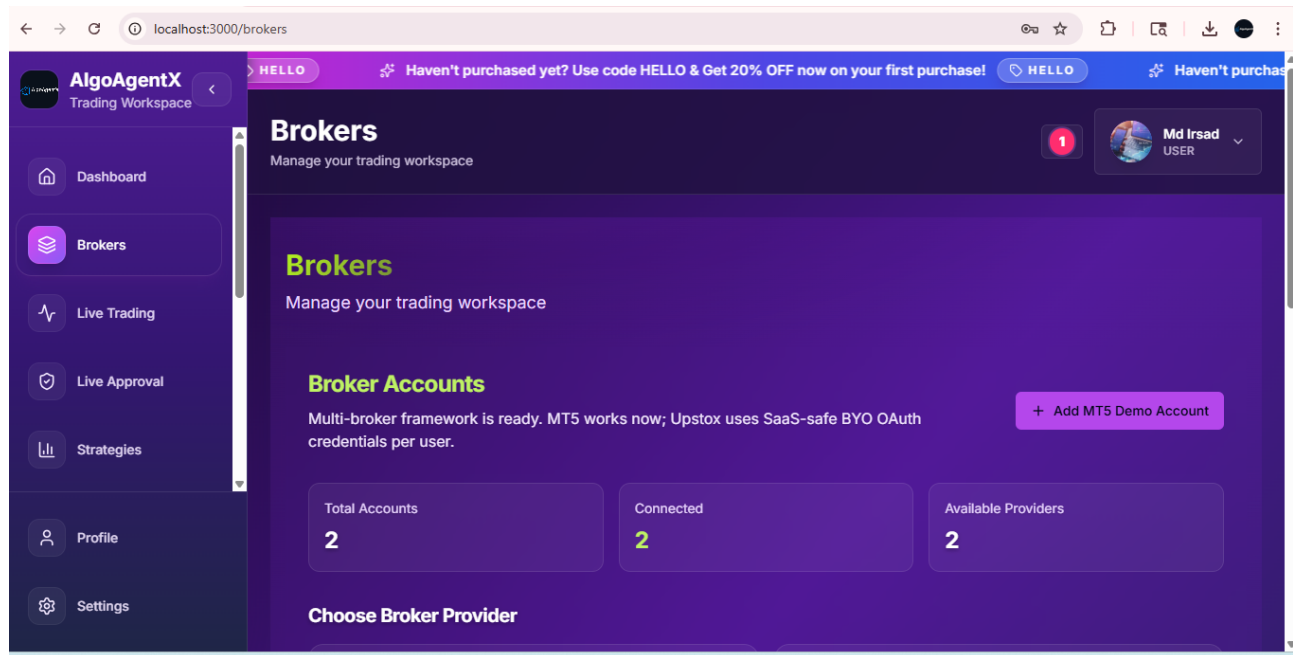
Live trading detail page with status, runner, broker sync, positions, and orders

Safety controls

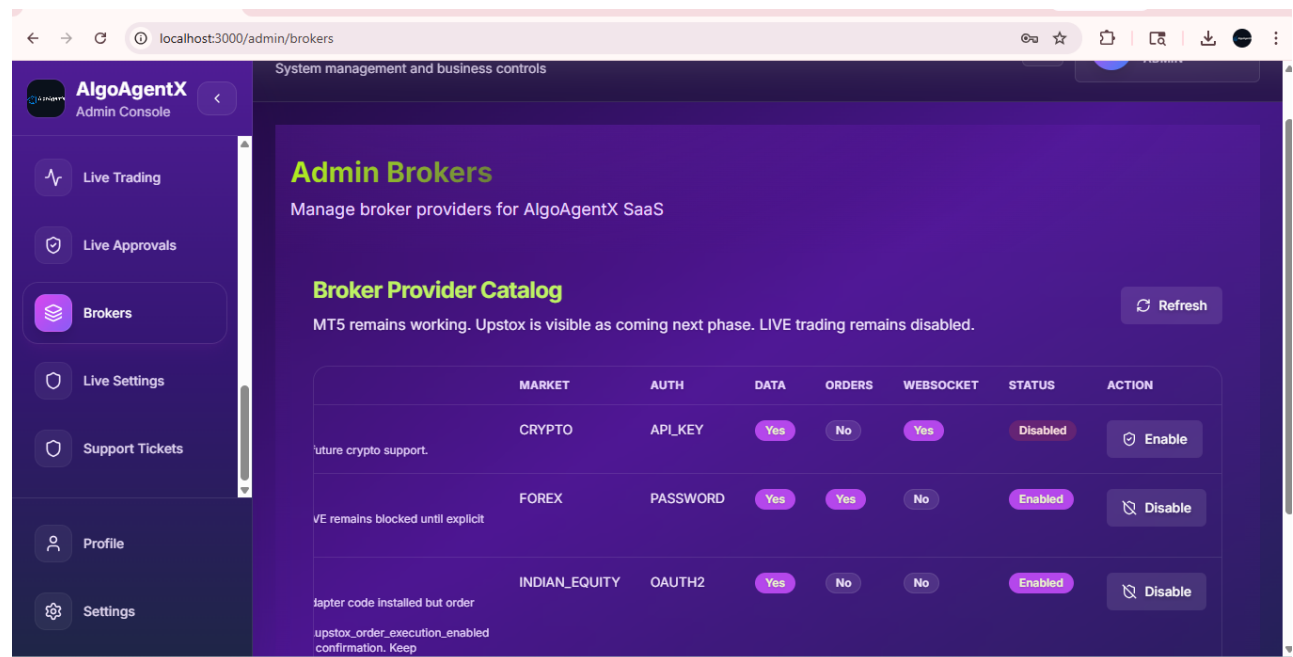
Readiness Checks Validate broker connection, market availability, instrument, account state, and deployment settings.	Risk Preview Preview order sizing, SL, TP, risk value, and expected execution parameters.	Duplicate Prevention Guard execution so repeated runner ticks do not create duplicate broker orders.
Broker Sync Refresh order status, open positions, realized/unrealized PnL, and account balance state.	Order Audit Store signal, order ID, status, timestamp, and execution diagnostics.	Operational Visibility Expose live engine status, runner state, latest activity, and deployment health.

9. Broker Integration Architecture

The broker layer is designed to separate broker connectivity from strategy logic. Cloud broker APIs can be connected directly, while MT5 desktop environments use an agent workflow to bridge local terminal execution with the SaaS backend.



User broker connection page



Admin broker and integration management

Broker connectivity model

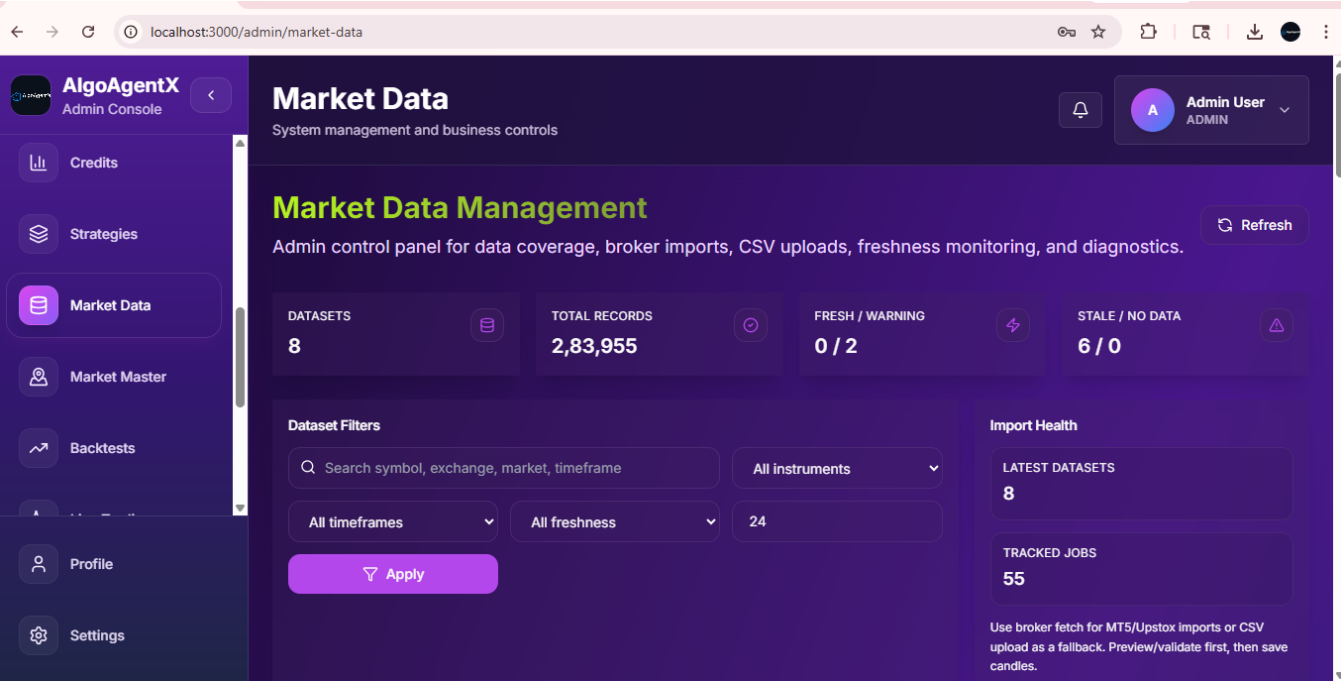
Cloud Broker APIs
Upstox-style authentication, account sync, market data access, and order routing.

MT5 Agent
Windows Agent connects local MetaTrader terminal with the SaaS backend using heartbeat and tokens.

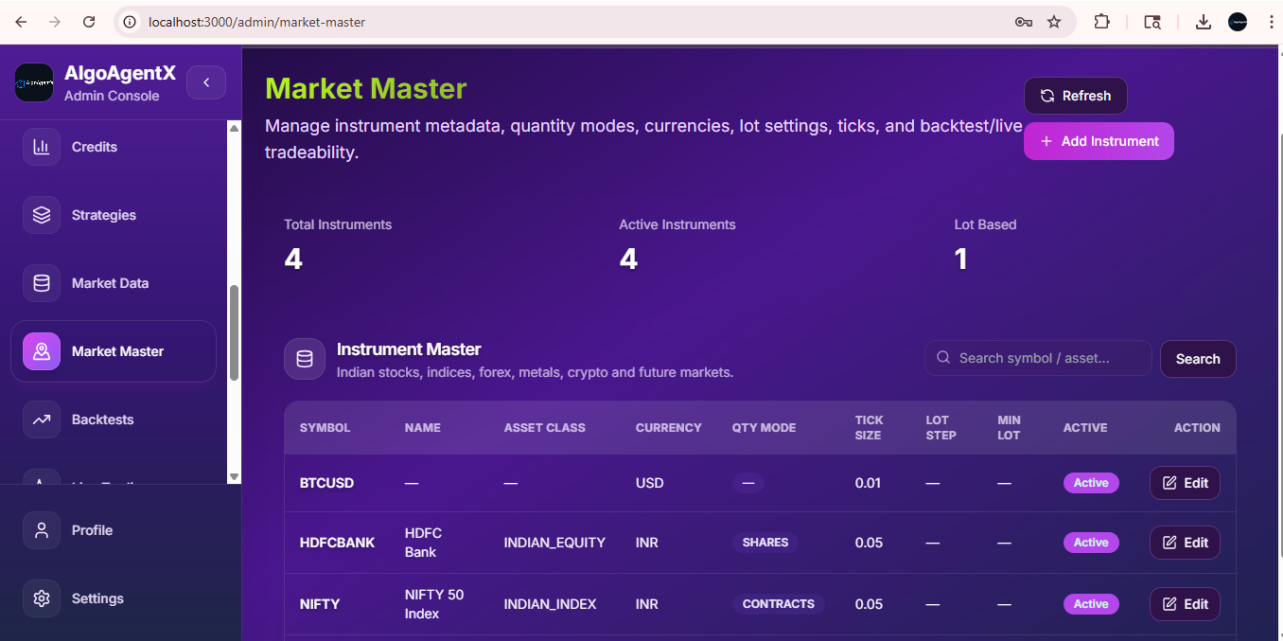
Account Sync
Balance, positions, symbols, and status information synchronized for deployment checks.

10. Market Data and Instrument Operations

Market data is a critical foundation for reliable backtesting and live readiness. The admin console supports market-data import, broker fetch workflows, symbol mapping, instrument specs, and coverage validation.



Admin market data import and fetch workspace



Market master and instrument configuration

Market-data controls

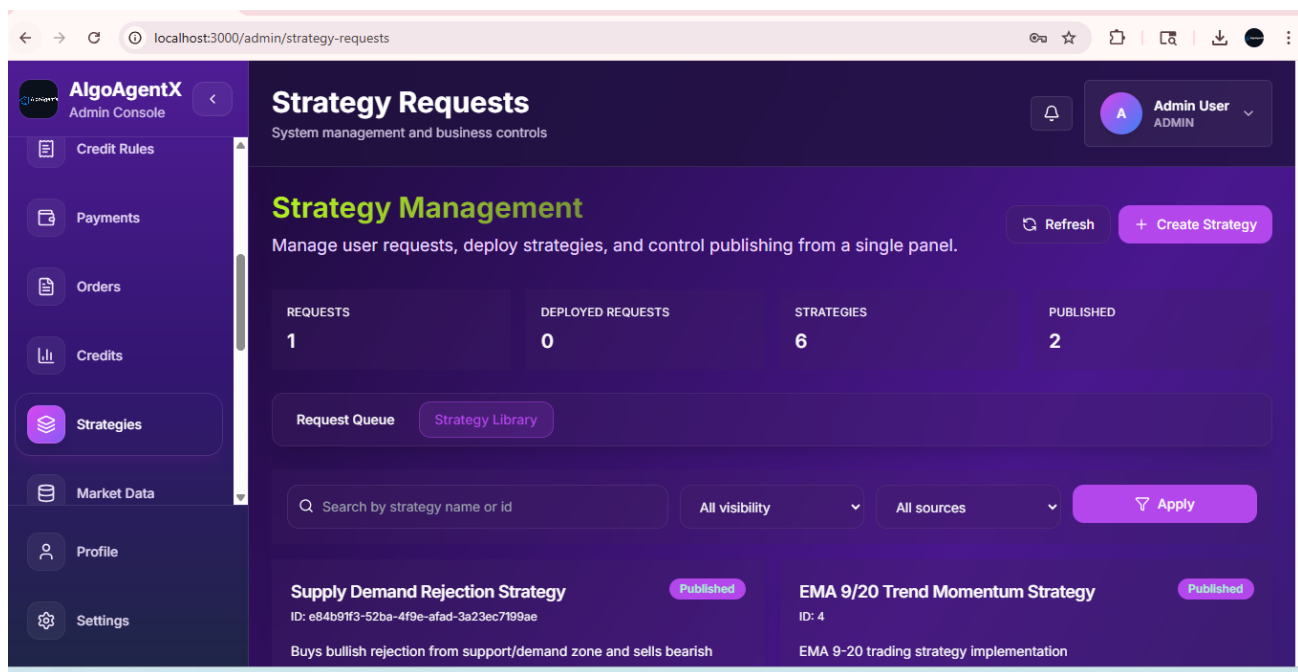
Data Import
CSV and broker fetch workflows for candle ingestion.

Symbol and Instrument Master
Per-broker symbols, tick size, lot size, quantity step, currency, and contract values.

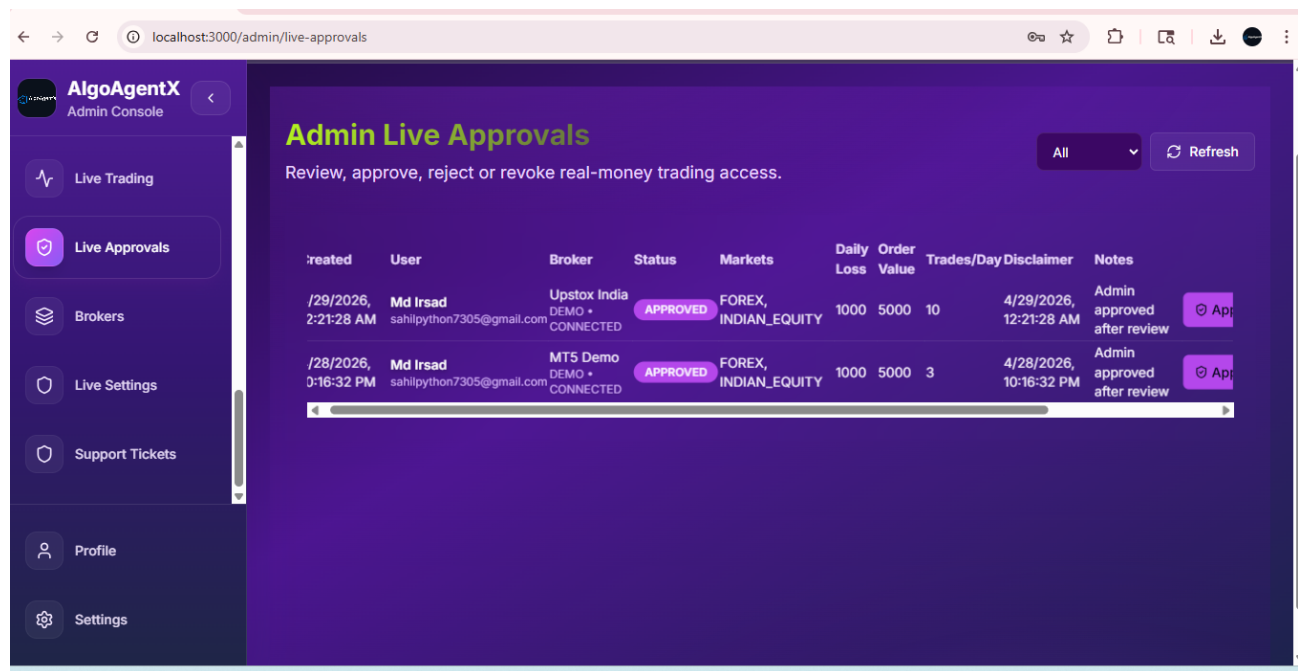
Coverage Validation
Backtests are blocked when selected data range is incomplete.

11. Admin Console and Platform Management

The admin console gives operators control over strategies, users, approvals, market data, instruments, subscriptions, payments, support, notifications, and settings. This turns the trading engine into a managed SaaS product instead of only a standalone trading script.



Admin strategy management workspace



Live trading approval and operational controls

Administration modules

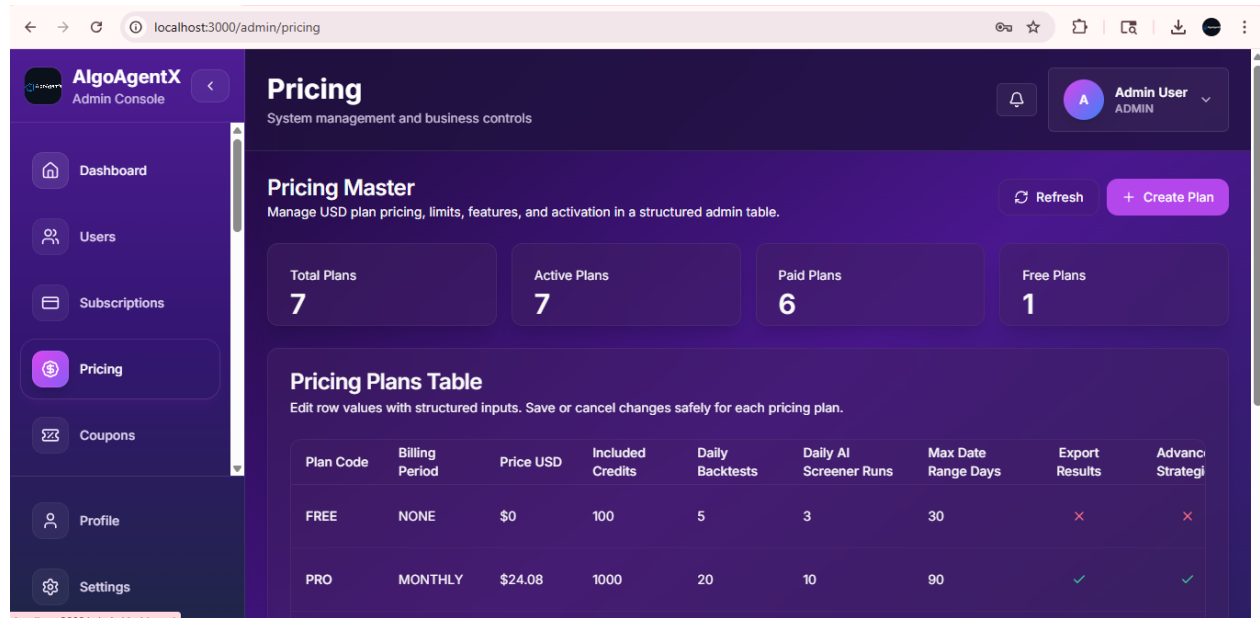
User Management
Accounts, roles, profiles, subscriptions, activity, and support visibility.

Strategy Management
Public templates, private strategies, development states, runtime defaults, and deployment control.

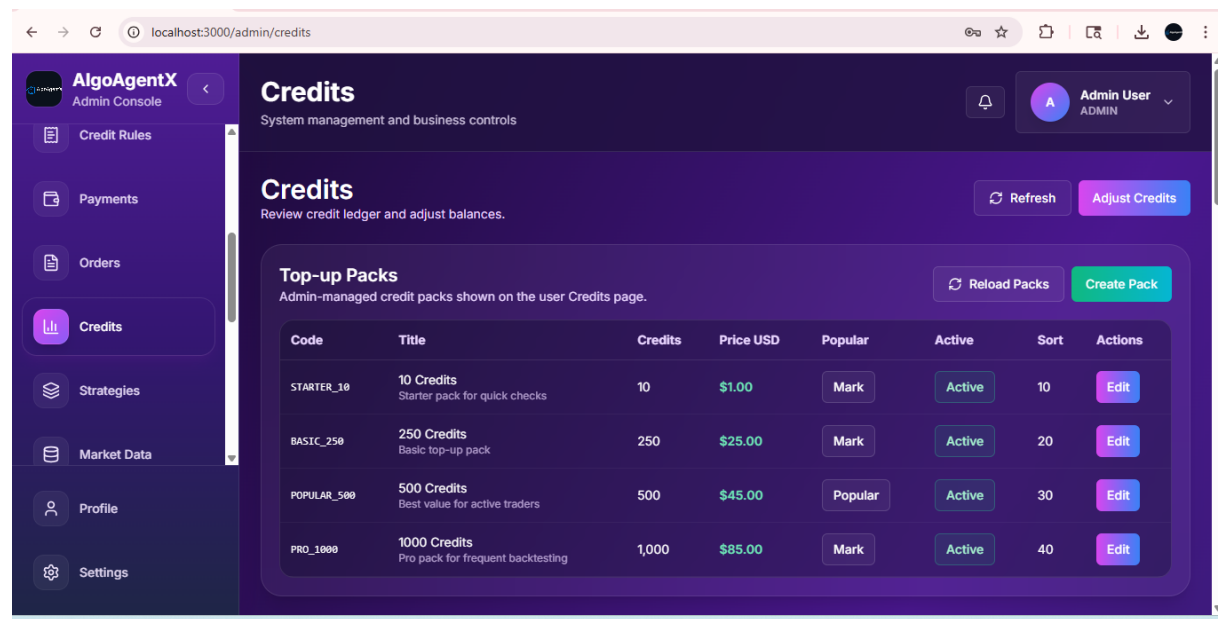
Live Approvals
Approval and safety workflow before broker-connected deployment.

12. SaaS Billing, Credits, and Subscription Operations

AlgoAgentX includes SaaS monetization modules for subscriptions, credits, coupons, payment tracking, orders, and billing administration. The credit model supports usage-based controls for compute-heavy workflows such as backtesting.



Admin pricing plan configuration



Credit management and usage operations

Commercial modules

Plans
Monthly/yearly subscription plans with pricing and feature limits.

Credits and Usage
Credit packs and usage rules for backtest execution and platform consumption.

Payments and Subscriptions
Orders, provider status, billing records, renewals, coupons, and subscription lifecycle.

13. Technical Stack

The platform uses a modern Python backend, relational database model, React/Next.js frontend, Docker deployment workflow, broker integrations, and analytics/reporting services.

Layer	Technology	Purpose
Backend	Python, FastAPI, SQLAlchemy, Pydantic	APIs, auth, validation, service orchestration, trading workflows
Database	PostgreSQL, async SQLAlchemy	Users, strategies, market data, trades, metrics, billing, audit records
Frontend	Next.js, React, Tailwind CSS	User workspace, admin console, dashboards, reports, forms
Trading	Backtest engine, risk engine, live runner, broker adapters	Signals, trades, position sizing, SL/TP, candle-close execution flow
Integrations	Broker APIs, MT5 Agent, Razorpay-style payment flow	Broker connectivity, local terminal bridge, payments and subscriptions
Operations	Docker, logging, monitoring, migrations, exports	Deployment, debugging, reporting, and platform stability

Engineering challenges solved

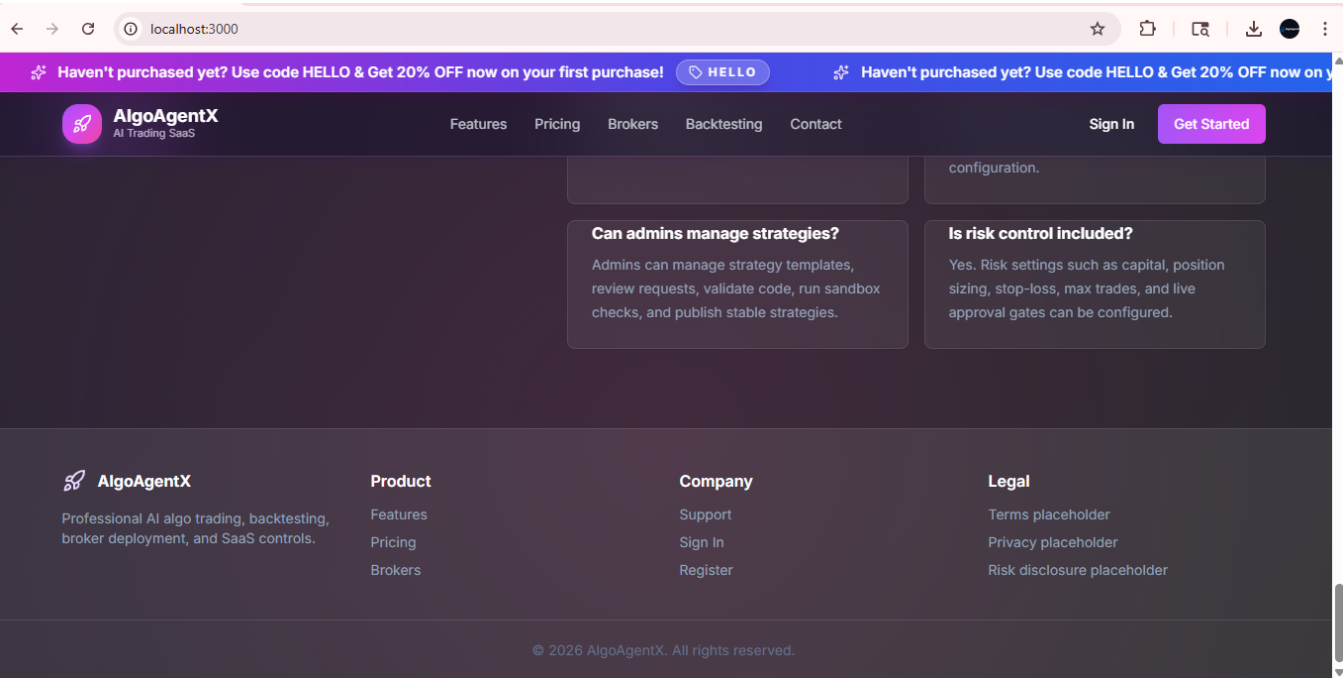
Data Coverage Prevented unreliable tests by blocking runs when market data is incomplete.	Runtime Flexibility Separated trade-management settings from strategy code for reusable strategy testing.	Live Timing Aligned runner behavior with completed candles instead of uncontrolled polling.
Broker Safety Added readiness checks, broker sync, duplicate-order prevention, and execution diagnostics.	SaaS Control Connected platform usage to plans, credits, coupons, subscriptions, and payments.	Admin Operations Provided operational controls for market data, users, strategies, approvals, and support.

14. Product Scope Summary

AlgoAgentX demonstrates a complete trading systems product workflow: from strategy configuration and backtest validation to broker-connected live readiness and SaaS administration. The platform combines trading logic, data validation, risk management, broker operations, analytics, reporting, billing, and admin tooling into one structured product.

Platform scope

Trading Engine Backtest engine, live runner, runtime settings, signal generation, risk rules.	Data Platform Market data ingestion, instrument master, symbol mapping, coverage checks.	Broker Layer Broker connection, account sync, MT5 Agent workflow, order routing.
Analytics Metrics, equity curve, drawdown, trade history, exports, reporting.	SaaS Platform Users, plans, credits, coupons, payments, subscriptions, support.	Admin Control Operations dashboard, market data, strategies, live approvals, settings.



Product footer and navigation overview

End of case study